

1 指令系统

1. ISA（指令集体系结构）规定的内容主要包括

- 指令格式、指令寻址方式、操作类型以及每种操作对应的操作数的相应规定
- 操作数的类型、操作数寻址方式以及是按大端方式还是小端方式存放
- 程序可访问的寄存器编号、个数和位数
- 存储空间的大小和编址方式
- 指令执行过程的控制方式等，包括程序计数器、条件码定义

2. 指令字长：一条指令所包含的二进制代码的位数，取决于操作码的长度、地址码的长度和地址码的个数。指令字长通常为字节的整数倍

- 单字长指令：指令字长等于机器字长的指令。访存 1 次就能将指令完整取出
- 半字长指令：指令字长等于半个机器字长的指令
- 双字长指令：指令字长等于两个机器字长的指令。需访存 2 次才能将指令完整取出

3. 扩展操作码：使操作码的长度随地址码的减少而增加，不同地址数的指令可具有不同长度的操作码

- 设地址长度为 n ，上一层留出 m 种状态，下层可扩展出 $m * 2^n$ 种状态

解析

指令字长 16bit，4 位为基本操作码

① 13 条三地址指令 $\Rightarrow$ 留出 $2^4 - 13 = 3$ 种

② 40 条二地址指令 $\Rightarrow$ 留出 $3 * 2^4 - 12 = 8$ 种

- 不允许短码是长码的前缀
- 各指令的操作码一定不能重复

1.1 指令的寻址方式

寻址方式：寻找指令或操作数有效地址的方式，即确定本条指令的数据地址及下一条待执行指令的地址的方法。分为**指令寻址**和**数据寻址**两大类

1.1.1 指令寻址

1. 指令寻址：寻找下一条将要执行的指令地址
2. 顺序寻址：通过程序计数器 PC 加 1（**1 条指令的长度**），自动形成下一条指令的地址
3. 跳跃寻址：由本条指令给出下条指令地址的计算方式。通过转移类指令实现
 - 绝对转移：地址码直接指出转移目标地址
 - 相对转移：地址码指出转移目的地址相对于当前 PC 值的偏移量

1.1.2 数据寻址

1. 数据寻址：如何在指令中表示一个操作数的地址，或怎样计算出操作数的地址
2. 寻址特征：指明属于哪种寻址方式
3. 形式地址 (A)：指令中的地址码字段并不代表操作数的真实地址
4. 有效地址 (EA)：形式地址结合寻址方式，可以计算出操作数在存储器中的真实地址，即 EA
5. 指令 = 操作码 + 地址码（操作数字段），地址码中可能包含寻址特征位和形式地址
6. 按字节编址，采用大端方式，操作数的机器数为 $ABCD00FFH$
 - ABH 对应一个字节，可以表示一个地址
 - 操作数占 $32bit = 4Byte$ ，即 4 个连续地址
 - 大端 $\Rightarrow$ 在内存中 FFH 表示的地址等于 ABH 的地址加 3
7. 内存地址是无符号数

解析

操作数采用基址寻址方式。基址寄存器的内容为 $F0000000H$ ，形式地址（补码表示）为 $FF12H$
内存地址无符号数，两者不能直接相加，故需要先转化为真值之后在进行加减

1.2 程序的机器级代码表示

通用寄存器					16bit	32bit	说明	
31	16	15	8	7	0			
		AH		AL		AX	EAX	累加器 (Accumulator)
		BH		BL		BX	EBX	基地址寄存器 (Base Register)
		CH		CL		CX	ECX	计数寄存器 (Count Register)
		DH		DL		DX	EDX	数据寄存器 (Data Register)
ESI						ESI		变址寄存器 (Index Register)
EDI						EDI		
EBP						EBP		堆栈基指针 (Base Pointer)
ESP						ESP		堆栈顶指针 (Stack Pointer)

1.2.1 常用指令

1. X86 指令集不允许两个操作数同时来自主存
2. **push** 指令：将操作数压入内存的栈，常用于函数调用
 - X86 默认以 $4B$ 为单位 $\Rightarrow$ 栈中元素固定为 $32bit$
 - 栈增长方向与内存地址增长方向相反。栈从高地址向低地址增长。即每压入一个元素， ESP 都减小
 - ESP 是栈顶，入栈前先将 ESP 值减 4，然后将操作数压入 ESP 指示的地址
3. **pop** 指令：出栈。出栈前先将 ESP 指示的地址中的内容出栈，然后将 ESP 的值加 4

	含义	有效地址	优点	缺点	备注
隐含寻址	不明显地给出操作数的地址，而是隐含操作数的地址		有利于缩短指令字长	需要增加存储操作数或隐含地址的硬件	
立即（数）寻址	指令字中的地址字段指出的是操作数本身，也称 立即数 ，采用补码表示	A 即是操作数	指令在执行阶段不访存，指令执行速度最快	A 的位数限制了立即数的范围	特征：#
直接寻址	指令字中的形式地址 (A) 就是操作数的真实地址 (EA)	$EA = A$	简单，指令在执行阶段仅需访存一次	A 的位数限制了该指令操作数的寻址范围，操作数的地址不易修改	
间接寻址	指令的地址字段是操作数有效地址所在主存单元的地址，即操作数地址的地址	$EA = (A)$	可扩大寻址范围，便于编制程序（用间接寻址可方便地完成子程序返回）	指令执行阶段要多次访存（ n 次间接寻址需 $n + 1$ 次访存）	
寄存器寻址	指令字中的地址字段给出的是操作数所在寄存器的编号	$EA = R_i$	执行阶段不用访存，执行速度快；寄存器数量远小于内存单元数，所以地址码位数较少，指令字长较短	CPU 的寄存器数量有限，寻址能力有限	
寄存器间接寻址	指令字中的 R_i 所指寄存器给出操作数所在主存单元的地址	$EA = (R_i)$	扩大了寻址范围，减少了访存次数		
相对寻址	将 PC 的内容加上指令格式中的形式地址 (A) 而形成操作数的有效地址，其中 (A) 是相对于下一条 PC 值的偏移量，补码表示	$EA = (PC) + A$	操作数的地址随 PC 值的变化而变化，且与指令地址之间总是相差一个固定的偏移量，因此便于程序浮动（一段代码在程序内部的浮动）		广泛应用于转移指令
基址寻址	将基址寄存器 (BR) 的内容加上指令字中的形式地址 (A) 而形成操作数的有效地址	$EA = (BR) + A$	扩大寻址范围（基址寄存器的位数大于形式地址 (A) 的位数）；有利于多道程序设计，并可用于编制浮动程序（整段代码在内存中浮动）		面向系统，用于为多道程序或数据分配存储空间
变址寻址	将变址寄存器 (IX) 的内容加上指令字中的形式地址 (A) 而形成操作数的有效地址	$EA = (IX) + A$	扩大寻址范围；适合编制循环程序		立足于用户，主要用于处理数组问题，变址寄存器的内容有用户设定。 地址 = $a[0]$ 地址 + $sizeof(<TYPE>) \times IX$
堆栈寻址					

4.

```
sub esp, 12 # 栈顶指针 - 12
add esp, 8 # 栈顶指针 + 8
```

5. **imul** 指令：有符号整数乘法指令。第一个操作数必须为寄存器

6. **idiv** 指令：有符号证书除法指令，只有一个操作数，即除数，被除数是 `edx:eax` 中的内容（共 64bit），操作结果有两部分：商和余数，商送到 `eax`，余数则送到 `edx`

7. **xor** 指令：逻辑异或操作指令（exclusive or）

8. **shl** 指令：逻辑左移指令（shift left）

9. **shr** 指令：逻辑右移指令（shift right）

10. **jcondition** 指令：条件转移指令，依据 CPU 状态字中的一系列条件状态转移

```
je <label> # jump when equal
jz <label> # jump when last result was zero
jne <label> # jump when not equal
jg <label> # jump when greater than
jge <label> # jump when greater than or equal to
jl <label> # jump when less than
jle <label> # jump when less than or equal to
```

11. **cmp** 指令：用于比较两个操作数的值，功能相当于 **sub** 指令，不保存操作结果，进根据运算结果设置 CPU 状态字中的条件码，与 **jcondition** 指令搭配使用

12. **test** 指令：对两个操作数进行逐位与运算，功能相当于 **and** 指令，不保存操作结果，进根据运算结果设置 CPU 状态字中的条件码，与 **jcondition** 指令搭配使用

13. **call** 指令：将下一条指令的地址（返回地址）入栈（压栈保存，在函数的栈帧顶部），然后无条件转移到由标签指示的指令，返回值放置在 `EAX` 中

14. **ret** 指令：实现子程序的返回机制，弹出栈中保存的指令地址，然后无条件转移到保存的指令地址

15. **enter** 指令：等价于

```
push ebp # 保存上一层函数的栈帧基址，即EBP旧值
mov ebp, esp # 设置当前函数的栈帧基址，即EBP新值
```

16. **leave** 指令：等价于

```
mov esp, ebp
pop ebp
```

17. 栈帧

- 栈帧最底部 $\Rightarrow$ 上一层栈帧地址 $\Rightarrow$ `EBP` 在上一层栈帧中的旧值
- `EBP`
- 局部变量：C 语言中越靠前定义的局部变量越靠近栈顶
- GCC 为保证数据的严格对齐而规定每个函数的栈帧大小必须是 16B 的倍数

- 调用参数: 参数列表中越靠前的参数越靠近栈顶。可以通过 `R[ebp] + 8` 或 `R[ebp] + 12` 获取实参
- 栈帧最顶部 $\Rightarrow$ 返回地址
- ESP

解析

```
804846a 39 c2 cmp dword ptr edx, eax
804846c 7e 0d jle xxxxxxxx
```

读出的信息:

18.
 - `39 c2` $\Rightarrow$ 每条指令占 $2B$
 - `804846c - 804846a = 2` 等于第一条指令长度 $\Rightarrow$ 按字节编址
 - `jle` 偏移量为 $0dH$
 - `jle` 采用 PC (EIP) 相对寻址, 若跳转, 则目标地址 = 下一条指令地址 + 偏移量 = $804846C + 2 + 0DH = 804847BH$

1.3 CISC & RISC

1. 复杂指令系统计算机 (CISC, Complex Instruction Set Computer): 增强原有指令的功能, 设置更为复杂的新指令实现软件功能的硬化
2. 精简指令系统计算机 (RISC, Reduce Instruction Set Computer): 减少指令种类和简化指令功能, 提高指令的执行速度

	CISC	RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般大于 200 条	一般小于 100 条
指令字长	不固定	定长
可访存指令	不加限制	只有 <code>LOAD/STORE</code> 指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差较大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成搞笑的目标代码程序	采用优化的编译程序, 生成代码较为高效
控制方式	绝大多数以微程序控制	绝大多数为组合逻辑控制, 效率较微程序高
指令流水线	可以通过一定方式实现	必须实现